

Nettoyage des données

Analyse des données — Séance 3

Stefania Marcassa

CY Cergy Paris Université — L3 Économie

Où nous allons aujourd'hui

1. **Reprise** : sélectionner, filtrer, vérifier. Vingt-cinq minutes.
2. Les valeurs manquantes : les trouver, et comprendre d'où elles viennent.
3. Décider quoi en faire — et assumer la décision.
4. Les doublons, qui ne sont pas tous des erreurs.

La première partie referme la séance précédente. La suite ouvre le vrai sujet.

Deux fichiers aujourd'hui : `cm02-communes.csv`, que vous connaissez, puis `cm03-salaires.csv`, que nous ouvrons ensemble.

Reprise — ce qu'on n'a pas eu
le temps de voir

Choisir des colonnes

```
df["POP"]                                # une colonne (Series)
df[["LIBGEO", "POP", "TAUX_CHOMAGE"]]    # plusieurs (DataFrame)
```

Les doubles crochets ne sont pas une coquetterie : vous passez une *liste* de noms. C'est la structure vue en séance 1.

```
df["CAT_URBAINE"].value_counts()
df["CAT_URBAINE"].value_counts(normalize=True)
```

`value_counts()` est le premier réflexe sur toute variable qualitative. Les quatre modalités affichées doivent correspondre à celles du dictionnaire — sinon, quelque chose ne va pas.

Filtrer des lignes

```
df[df["POP"] > 100000]
```

La condition à l'intérieur produit une suite de vrai/faux, un par ligne. Le tableau ne conserve que les lignes vraies.

```
df[(df["POP"] > 50000) & (df["TAUX_CHOMAGE"] > 18)]      # et  
df[(df["CAT_URBAINE"] == "1") | (df["CAT_URBAINE"] == "2")] # ou  
df[df["CODGEO"].str[:2].isin(["75", "93", "95"])]        # appartenance
```

Les parenthèses autour de chaque condition sont obligatoires. Sans elles, Python évalue dans le mauvais ordre et renvoie une erreur peu explicite.

Trier pour repérer une anomalie

```
df.sort_values("NB_ENTREPRISES", ascending=False).head(15)
```

`ascending=False` trie du plus grand au plus petit. Douze communes dépassent 9 999 établissements : les deux valeurs suspectes n'apparaissent qu'aux rangs 13 et 14. **Un effectif brut classe d'abord les grandes villes.**

```
df["NB_ENTREPRISES"].value_counts().head(3)    # 9999 : 2 fois
```

Plus direct. Dans une variable continue, une valeur qui revient deux fois est suspecte : Douai, 39 508 habitants, et Saint-Junien, 11 245 habitants, affichent le même effectif exact.

On ne calcule rien avant d'avoir lu le dictionnaire. Il indique 9 999 = non-réponse. La conversion vient d'abord, la moyenne ensuite.

Compter, avant et après

```
len(df)                # 45
df["TAUX_CHOMAGE"].isna().sum()  # 0
```

Zéro manquant, et six communes sans taux de chômage. Tant que la colonne est lue en texte, les six "nd" sont des modalités comme les autres, et `isna()` ne voit rien.

```
df["TAUX_CHOMAGE"] = pd.to_numeric(
    df["TAUX_CHOMAGE"].str.replace(",", ".", regex=False),
    errors="coerce")
```

```
df["TAUX_CHOMAGE"].isna().sum()                # 6
print(len(df), "->", len(df[df["POP"] > 100000]))  # 45 -> 16
```

Comptez avant et après chaque opération. Un compte qui ne bouge pas quand il devrait est le signal le moins cher du métier.

Trois vérifications avant de quitter le fichier

1. `df.shape` correspond-il à ce qu'annonce la documentation ?
2. `df.info()` : chaque colonne a-t-elle le type attendu ?
3. `value_counts()` sur les variables qualitatives : les modalités correspondent-elles au dictionnaire ?

Ces trois contrôles prennent deux minutes. Ils vous éviteront la moitié des erreurs du semestre — et ils constituent une question type du contrôle continu.

Vous savez charger un fichier, comprendre ce qu'il contient, en extraire ce qui vous intéresse et vérifier que vous ne vous êtes pas trompée.

Il reste le travail que personne ne montre : **rendre ces données utilisables**.

Le principe de la suite

Le nettoyage n'est pas une opération technique préalable à l'analyse. C'est une suite de **décisions**, dont chacune doit pouvoir se justifier — et dont chacune modifie le résultat.

Les valeurs manquantes

Les trouver

```
df.isna().sum()           # effectif par colonne  
df.isna().mean().round(3) # part par colonne
```

Sur le fichier des communes, une fois les conversions de la séance 2 faites :

Colonne	Manquants	Part
TAUX_CHOMAGE	6	13,3 %
NB_ENTREPRISES	2	4,4 %
Les cinq autres	0	0 %

Deux colonnes seulement, et sur quarante-cinq lignes. C'est un fichier de cours. La question qui suit, elle, est la même sur tous les fichiers.

La question qui compte : *qui* sont les manquants ?

```
df.groupby("CAT_URBAINE")["TAUX_CHOMAGE"].apply(  
    lambda s: s.isna().sum())
```

Catégorie urbaine	Communes	Sans taux	Part
1 — grand pôle	13	0	0 %
2 — couronne	16	0	0 %
3 — petit pôle	10	0	0 %
4 — rural	6	6	100 %

Les six valeurs absentes sont les six communes rurales, et elles seulement.

L'absence n'est pas un accident : elle est produite par le secret statistique, qui dépend de la taille de la commune. Une variable que vous observez explique parfaitement ce qui manque.

Les manquants qui ne ressemblent pas à des manquants

Codage	Exemple	Symptôme
Code numérique	9, 99, 999, 9 999	Un effectif d'entreprises trop rond
Texte	"nd", "NR", "-"	Colonne numérique lue en texte
Chaîne vide	" "	Comptée comme une modalité
Valeur impossible	-1, 0, 1900	Aucun symptôme

Vous avez rencontré les deux premiers en séance 2 : "nd" dans TAUX_CHOMAGE, et 9 999 dans NB_ENTREPRISES.

Pandas ne les reconnaît pas : `isna()` renvoie zéro manquant, et tous les calculs fonctionnent.

Seul le dictionnaire des variables vous dira comment la non-réponse est codée.

Un second fichier

Quarante-cinq communes ne suffisent pas pour la suite. Nous passons à des données individuelles.

```
brut = pd.read_csv("cm03-salaires.csv",
                   encoding="latin-1", sep=";", decimal=".", thousands=" ",
                   dtype={"IDENT": str, "SECTEUR": str, "REGION": str})
df = brut.copy()
print(brut.shape)           # (8012, 15)
```

Huit mille douze lignes, quinze colonnes. **Une ligne est un poste occupé dans l'année**, pas une personne : retenez-le, nous y reviendrons.

Données fabriquées pour l'enseignement. Ordres de grandeur plausibles, valeurs fictives. Le dictionnaire est sur le site du cours.

Le calcul qui fonctionne, et qui est faux

```
df["SALAIRE_NET"].mean() # 67745.36
df["SALAIRE_NET"].value_counts().head(4)
```

Aucune erreur, aucun avertissement sur la moyenne. La colonne, elle, parle :

Valeur	Effectif	Ce que dit le dictionnaire
0	675	Sans objet : non-salariés
999 999	525	Non-réponse
10 000	72	Plafond de diffusion
1 859,27	3	Rien : trois salariés au même euro près

Trois problèmes dans une seule colonne, et une quatrième ligne qui n'en est pas un : sur 8 012 lignes, 98,9 % des salaires ne reviennent jamais. C'est l'écart d'ordre de grandeur qui trahit un code, pas la répétition seule.

Trois origines, trois conséquences

Manquant complètement au hasard

L'absence ne dépend de rien. Un questionnaire égaré, une ligne corrompue.

Supprimer les lignes réduit la précision, mais ne biaise pas.

Manquant au hasard, conditionnellement

L'absence dépend de variables *observées*. Le taux de chômage manque pour les communes rurales, et vous observez la catégorie urbaine.

Supprimer biaise ; on peut corriger en tenant compte de la variable en cause.

Manquant non au hasard

L'absence dépend de la valeur *elle-même*. Les hauts revenus ne déclarent pas leur revenu.

Aucune correction n'est possible à partir des seules données observées.

Des questions ?